14、触摸屏驱动

全志V3S嵌入式驱动开发（触摸屏驱动）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2024-06-28 21:19:37 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量1.8k Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 6

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文介绍了全志V3S嵌入式平台上的触摸屏驱动开发过程，包括查看lcd电路图，理解设备树中i2c与TP信号的配置，查看驱动代码，下载安装tslib库进行测试验证。通过分析和测试，确保触摸屏驱动的正确移植与功能实现。

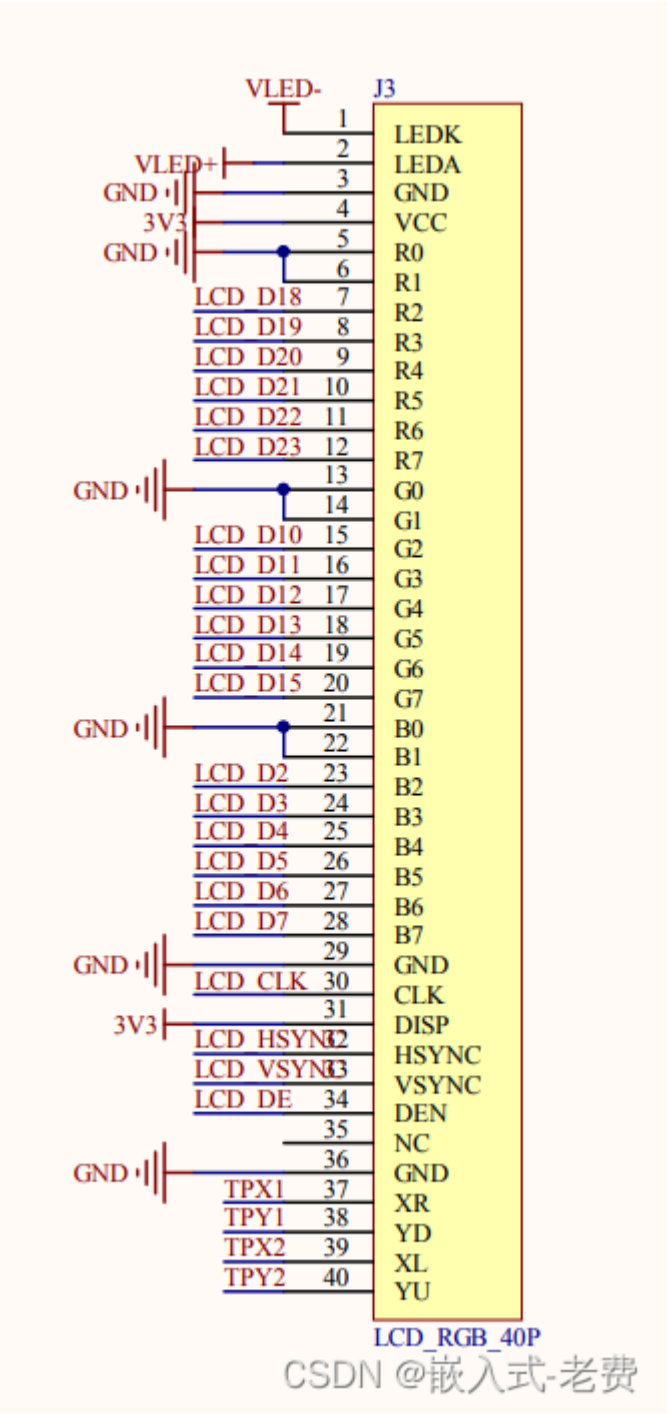
摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

所谓的触摸屏，其实就是在普通的lcd屏幕之上，再加一层屏而已。这个屏是透明的，这样客户就可以看到下面lcd屏幕的内容，另外一方面，通过触摸信号，还可以实现人和机器的自然交互，这是一种非常好的体验形式。像早期的按键手机，大家用着其实还行，只不过后来转成智能手机，只剩下一个屏幕之后，就一股脑全转成触摸屏手机了。

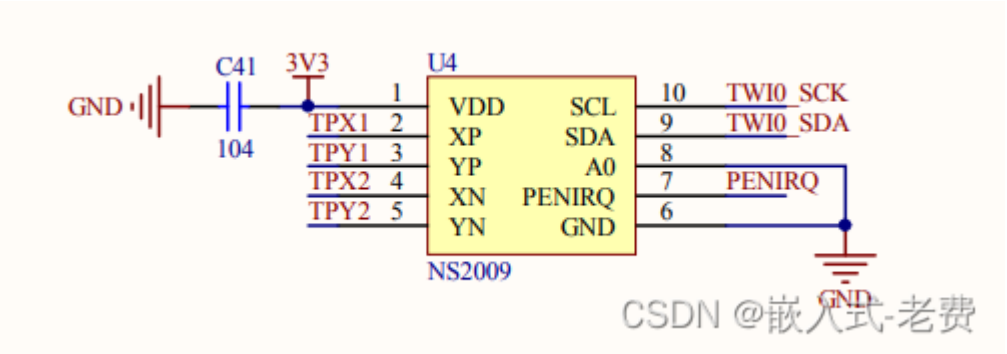
1、查看lcd电路图

Open: 350955b3b434ba010f597b774744a0fa_MD5.png



之前我们说过这个lcd 40 pin的原理图。上次分析的时候，卖了一个关子。就是这个TPX1、TPY1、TPX2、TPY2，四根信号线。这四根线并没有直接连接到soc上面，而是先连接到ns2009，然后再通过i2c的形式连接到soc上面。

Open: 54eef0f92ecf05134328bd4723924315_MD5.png



简单浏览下，发现TP信号连接到了2、3、4、5号pin上面，soc需要获取触摸信息的话，通过SCK & SDA就可以，当然这里还有一个PENIRQ，理论上是可以通过中断的形式获取触摸信息的。但是，实际看linux驱动代码的话，发现还是定时轮询的形式。

此外选择lcd屏幕的时候，一定要选择带有触摸屏的那种，千万要和商家沟通清楚。不然白白浪费了自己的时间。

2、设备树

上面分析电路得出结论，soc是通过i2c获取按键信息的，那么可以直接查找sun8i-v3s-licheepi-zero.dts，

```
&i2c0 {
    status = "okay";

    ns2009: ns2009@48 {
        compatible = "nsiway,ns2009";
        reg = <0x48>;
    };
};
```

引脚信息可以在sun8i-v3s.dtsi查找到，分别是PB6、PB7

```
        i2c0_pins: i2c0 {
            pins = "PB6", "PB7";
            function = "i2c0";
        };
```

更详细的寄存器、中断、时钟信息也可以在这个文件的后半部分找到，

```
    i2c0: i2c@01c2ac00 {
        compatible = "allwinner,sun6i-a31-i2c";
```

```
            reg = <0x01c2ac00 0x400>;
            interrupts = <GIC_SPI 6 IRQ_TYPE_LEVEL_HIGH>;
            clocks = <&ccu CLK_BUS_I2C0>;
            resets = <&ccu RST_BUS_I2C0>;
            pinctrl-names = "default";
            pinctrl-0 = <&i2c0_pins>;
            status = "disabled";
            #address-cells = <1>;
            #size-cells = <0>;
        };
```

上面说的这些内容都是默认配置好的，只要是linux 4.14.y内核，就不需要自己再动手配置一遍了。

3、查看驱动代码

因为这里使用到了i2c协议，也使用到了ns2009芯片，所以只需要在drivers/input/touchscreen下面查找下，有没有类似于ns2009的驱动就好。观察发现，果然有一个ns2009.c文件，里面内容也和我们想的差不多，

```
static const struct i2c_device_id ns2009_ts_id[] = {
    { "ns2009", 0 },
    { }
};
MODULE_DEVICE_TABLE(i2c, ns2009_ts_id);

static struct i2c_driver ns2009_ts_driver = {
    .probe = ns2009_ts_probe,
    .id_table = ns2009_ts_id,
    .driver = {
        .name = NS2009_TS_NAME,
    },
};
module_i2c_driver(ns2009_ts_driver);
```

这部分代码因为是包含在zImage里面的，因此也就不需要重新编译、重新烧入了。和其他驱动相比较，这算是很方便的一种情形。

4、下载、安装tslib库

tslib基本就是触摸屏调试的一个标配。它不仅仅可以帮助我们对触摸屏做出一些参数标定，还可以用它做接口，做一些测试或者定制应用开发。首先，我们需要下载一个tslib代码，

```
https://github.com/libts/tslib/releases
```

选择好版本之后，就可以拷贝到ubuntu虚拟机，准备用交叉编译工具开始编译，

```
mkdir __install
./configure -host=arm-linux-gnueabihf -prefix=$PWD/__install
make
make install
```

编译安装好之后，所有的文件都被安装到__install目录下面。接下来要做的，就是把这些文件拷贝到sd卡的ext4系统上面去，

```
cp bin/* /media/feixiaoxing/64959927-1d53-4bb1-8102-d7c16fc29ade/bin/
cp lib/*.so* /media/feixiaoxing/64959927-1d53-4bb1-8102-
d7c16fc29ade/lib/ -rfdv
cp lib/ts/ /media/feixiaoxing/64959927-1d53-4bb1-8102-d7c16fc29ade/lib/
-rfdv
cp etc/ts.conf /media/feixiaoxing/64959927-1d53-4bb1-8102-
d7c16fc29ade/etc/
```

自此，tslib就安装好了，可以准备测试了。

5、测试验证tslib

测试验证之前，需要先配置一下环境变量，有几个参数需要设置下，

```
export set TSLIB_TSDEVICE=/dev/input/event0
export set TSLIB_CONFFILE=/etc/ts.conf
export set TSLIB_PLUGINDIR=/lib/ts
export set TSLIB_CALIBFILE=/etc/pointercal
export set TSLIB_CONSOLEDEVICE=none
export set TSLIB_FBDEVICE=/dev/fb0
```

第一个参数表示event设备节点的位置。如果ns2009驱动正常安装，肯定会有这个节点的。第二个参数表示配置文件的位置。第三个参数表示插件的位置。第四个参数表

示标定参数保存的位置。第五个参数可以不设置，直接写成none。第六个参数表示屏幕节点的位置。

当然测试的时候，别忘记点亮屏幕，屏幕不点亮啥也做不了，

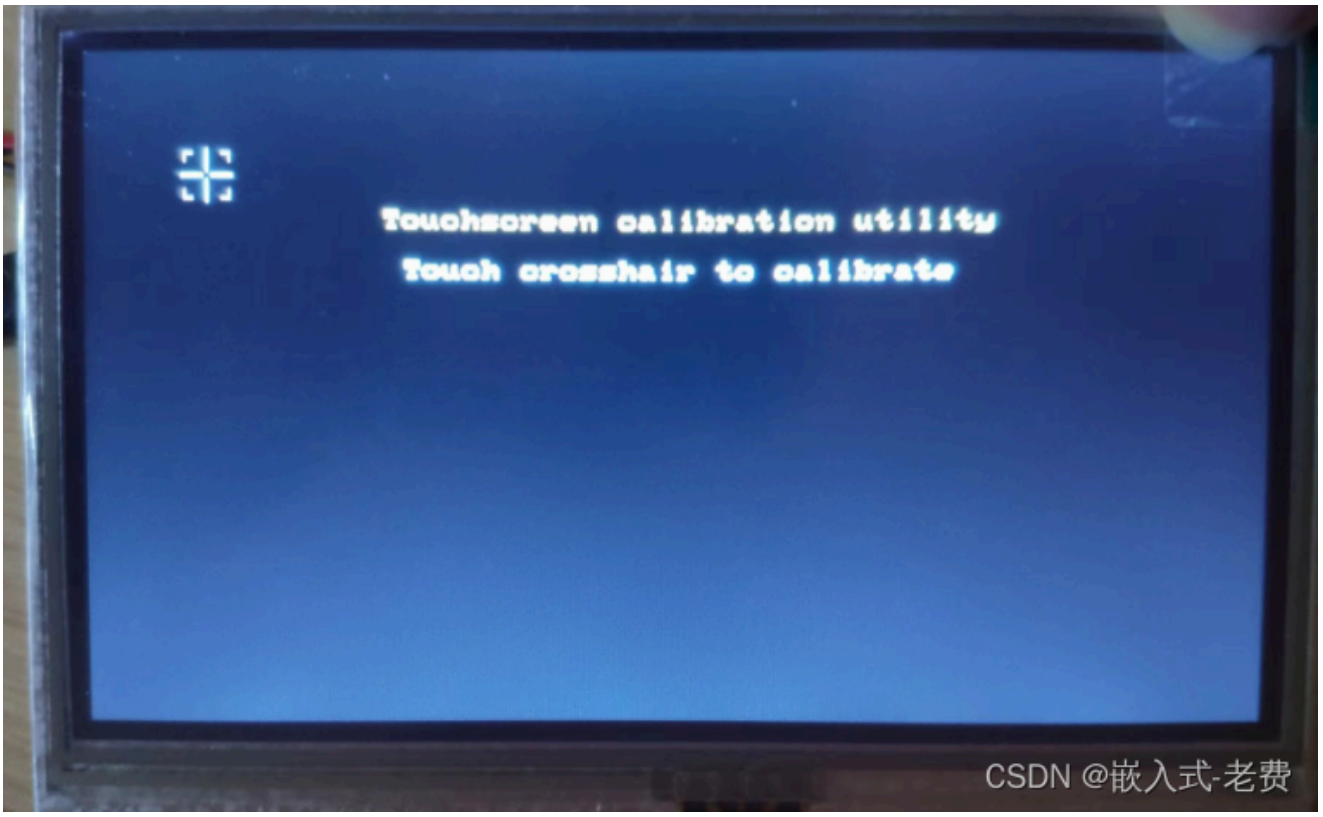
```
echo 34 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio34/direction
echo 1 > /sys/class/gpio/gpio34/value
```

首先，先用ts_calibrate做一下触摸屏的标定，

```
# ts_calibrate
xres = 480, yres = 272
```

不出意外，屏幕上可以看到这样的情形，

Open: d58b6325beacaa93545283bd35689eec_MD5.png



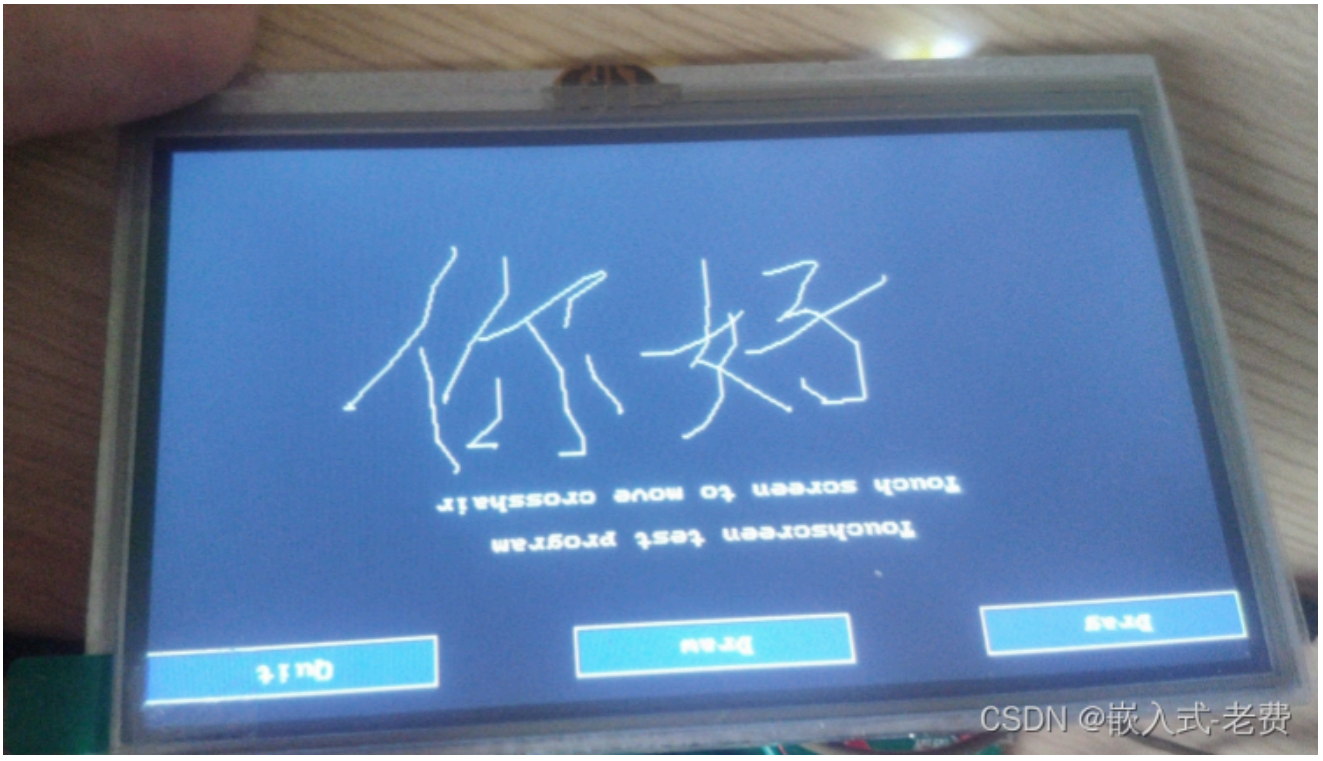
有一个十字箭头的东西会依次出现在四个角落和中间位置，每点击依次，它就会跳到新的位置。五次都结束以后，标定就结束了，

```
# ts_calibrate
xres = 480, yres = 272
[  113.404260] random: crng init done
```

```
Took 1 samples...
Top left : X =  534 Y = 1156
Took 3 samples...
Top right : X = 3496 Y = 1334
Took 2 samples...
Bot right : X = 3510 Y = 3270
Took 2 samples...
Bot left : X =  453 Y = 3075
Took 1 samples...
Center : X = 1963 Y = 2066
-15.522461 0.126093 0.002039
-46.474915 -0.005545 0.088761
Calibration constants: -1017280 8263 133 -3045780 -363 5817 65536
```

接下里就可以用ts_test开始测试了，

Open: 9860b5c60c183c30cbdab075435747a7_MD5.png



测试有两种方式，一种是拖着十字到处移动，还有一种就是画图。如上图所示，这就是画图的测试方法，只不过我们写了两个汉字。不管是那种方式，需要先到对应的按钮选中一下。因为电阻屏反应没那么灵敏，等按下去的时候，时间多停留一会，直到按钮颜色变化之后，再开始测试。如果能写出自己想要的内容，基本表明驱动移植成功。

串口方面，也可以看到这些打印信息，

```
# ts_test
1178.709275:    265     77    255
1178.748307:    265     77      0
1178.789301:    265     42    255
1178.829277:    508    283    255
1178.868270:    508    283      0
1179.549275:    268     15    255
1179.589293:    284     25    255
1179.629289:    297     38    255
```